

FreeRTOS Lab 2

Task Monitor

OS lab email: oslab@mail.csie.ncku.edu.tw

TA email: p76144613@gs.ncku.edu.tw



Operating Systems &
Embedded Systems Lab

OUTLINE

01 Lab requirement

02 UART

03 Lab 2

Lab requirement

- Create **4 tasks** (Don't need to write by yourself. The complete codes are included in the upcoming reference slides.)
 - Red_LED_App 、 Green_LED_App 、 Delay_App 、 TaskMonitor_App
- **Taskmonitor()** is called periodically by **TaskMonitor_App**.
- **Taskmonitor()**
 - **Traverse** ReadyTaskList, DelayedTaskList, OverflowDelayedTaskList.
 - **Print** TCB information by **UART**
 - **Task Name, Priority(Base/actual), Stack Pointer, Top of stack Pointer, and Task State.**

Lab requirement – Demo result

1. Periodically display the TCB (Task Control Block) information for the 5 active tasks, including: **Task Name, Priority(Base/actual), Stack Pointer, Top of stack Pointer, and Task State.**
2. TCB information needs to be correct.

5 tasks

com6 (CONNECTED)					
Name	Priority(Base/actual)	pxStack	pxTopOfStack	State	
IDLE	0 /0	0x20000f10	0x200010bc	Ready	}
TaskMonitor	3 /3	0x20000840	0x20000bcc	Ready	
Red_LED	1 /1	0x20000370	0x200004f4	Blocked	}
Green_LED	1 /1	0x200005d8	0x2000075c	Blocked	
Delay_App	14/14	0x20000ca8	0x20000e2c	Overflow	}
Name	Priority(Base/actual)	pxStack	pxTopOfStack	State	
IDLE	0 /0	0x20000f10	0x200010bc	Ready	}
TaskMonitor	3 /3	0x20000840	0x20000bcc	Ready	
Green_LED	1 /1	0x200005d8	0x2000075c	Blocked	}
Red_LED	1 /1	0x20000370	0x200004f4	Blocked	
Delay_App	14/14	0x20000ca8	0x20000e2c	Overflow	}
Name	Priority(Base/actual)	pxStack	pxTopOfStack	State	
IDLE	0 /0	0x20000f10	0x200010ac	Ready	}
TaskMonitor	3 /3	0x20000840	0x20000bcc	Ready	
Red_LED	1 /1	0x20000370	0x200004f4	Blocked	}
Green_LED	1 /1	0x200005d8	0x2000075c	Blocked	
Delay_App	14/14	0x20000ca8	0x20000e2c	Overflow	}
Name	Priority(Base/actual)	pxStack	pxTopOfStack	State	
IDLE	0 /0	0x20000f10	0x200010b4	Ready	}
TaskMonitor	3 /3	0x20000840	0x20000bcc	Ready	
Red_LED	1 /1	0x20000370	0x200004f4	Blocked	}
Green_LED	1 /1	0x200005d8	0x2000075c	Blocked	
Delay_App	14/14	0x20000ca8	0x20000e2c	Overflow	}
Name	Priority(Base/actual)	pxStack	pxTopOfStack	State	
IDLE	0 /0	0x20000f10	0x200010b4	Ready	}
TaskMonitor	3 /3	0x20000840	0x20000bcc	Ready	
Red_LED	1 /1	0x20000370	0x200004f4	Blocked	}
Green_LED	1 /1	0x200005d8	0x2000075c	Blocked	
Delay_App	14/14	0x20000ca8	0x20000e2c	Overflow	}

In ReadyTaskList

In DelayedTaskList

In OverflowDelayedTaskList

Lab requirement – Lab 2 grading

- **Demo (5%)**

- (2%) Print 5 tasks.
- (2%) TCB details are correct and align with the demo results.
- (1%) Don't print out other unrelated text.

- **Lab Report (3%)**

- Write according to the format given on Moodle.
 - Please submit a PDF file with the file name as **studentID_labNo.pdf**
 - e.g., P76144613_lab2.pdf
- The Report must be uploaded on Moodle before **23:59 on 4/16(Thur.)**; submissions after the deadline will not be graded.

OUTLINE

01 Lab requirement

02 UART

03 Lab 2

UART: USB-to-TTL serial adapter



GND



TXD (Transmit)



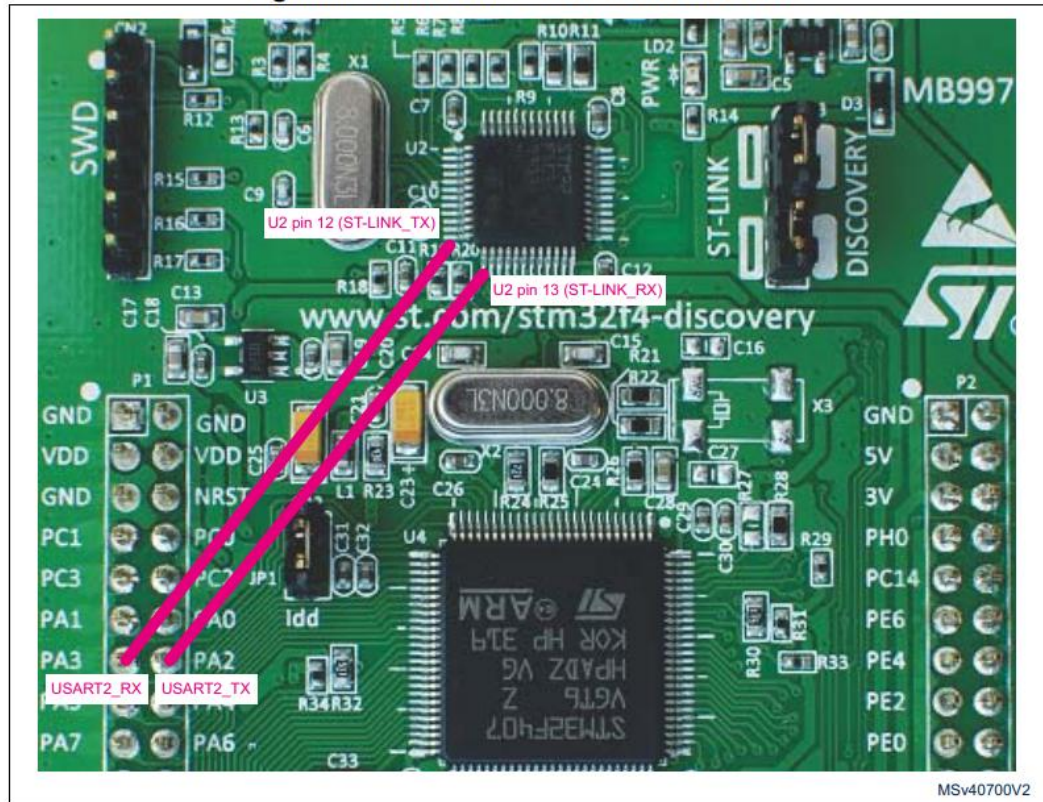
RXD (Receive)



3.3V/5V

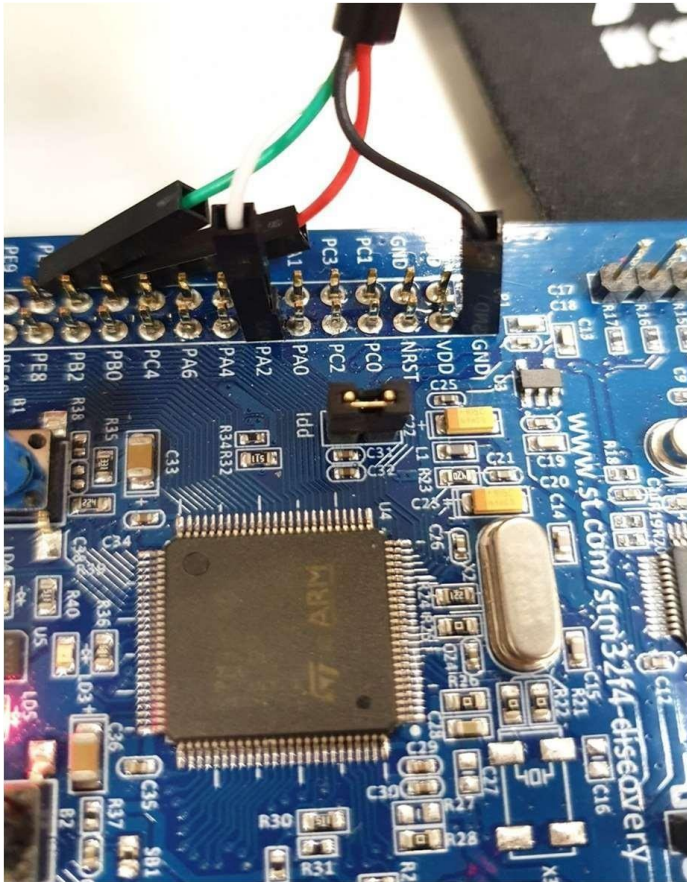
USART PIN

Figure 7. ST-LINK VCP connection to USART2



Using an USART to USB dongle from the market connected for instance to STM32F407 USART2 available on connector P1 pin 14 (**PA2**: USART2_TX) and P1 pin 13 (**PA3**: USART2_RX).

Connection



F407G-DISC transmits data(TXD) to PC.

Therefore, we need to use **PA2**(TXD).

PC will receive external data(RXD), so we need to use **white line**(RXD) connecting to PA2(TXD).

We will only use the **black** and **white** lines of the USB-to-TTL Serial Adapter.



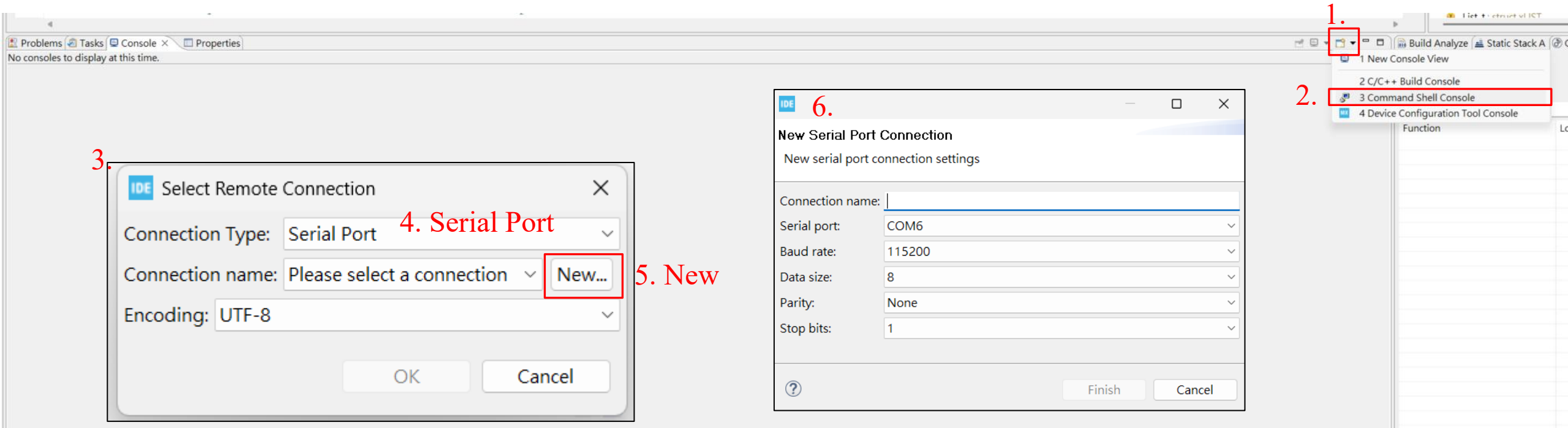
USART Set Up

The screenshot displays the STM32CubeMX Pinout & Configuration window for an STM32F407VGTx LQFP100. The interface is divided into several sections:

- Categories (Left):** A list of peripheral categories. Red arrow 1 points to the 'Connectivity' category. Red arrow 2 points to 'USART2' in the list.
- USART2 Mode and Configuration (Center):**
 - Mode:** A dropdown menu set to 'Asynchronous'. Red arrow 3 points to this dropdown.
 - Hardware Flow Control (RS232):** Set to 'Disable'.
 - Configuration:** Includes tabs for GPIO Settings, NVIC Settings, DMA Settings, Parameter Settings, and User Constants. The 'Parameter Settings' tab is active.
 - Basic Parameters:** Baud Rate: 115200 Bits/s, Word Length: 8 Bits (including Parity), Parity: None, Stop Bits: 1.
 - Advanced Parameters:** Data Direction: Receive and Transmit, Over Sampling: 16 Samples.
- Pinout View (Right):** A visual representation of the microcontroller pins. Red circles highlight the assignments:
 - USART2_TX:** Assigned to PA2.
 - USART2_RX:** Assigned to PA3.

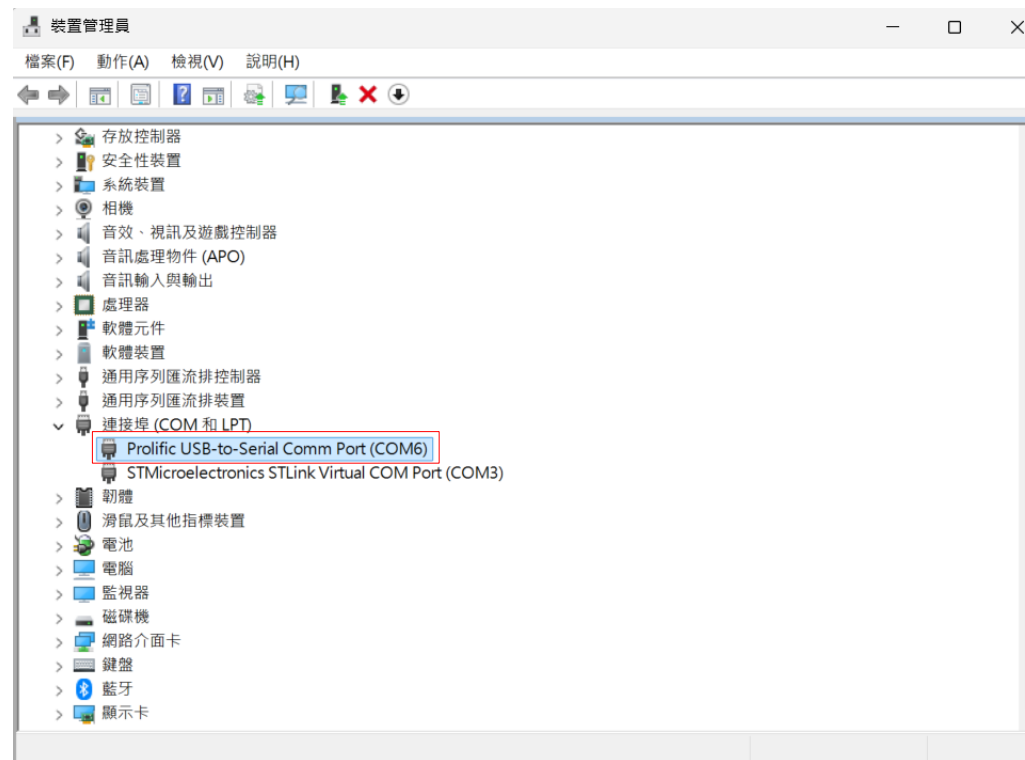
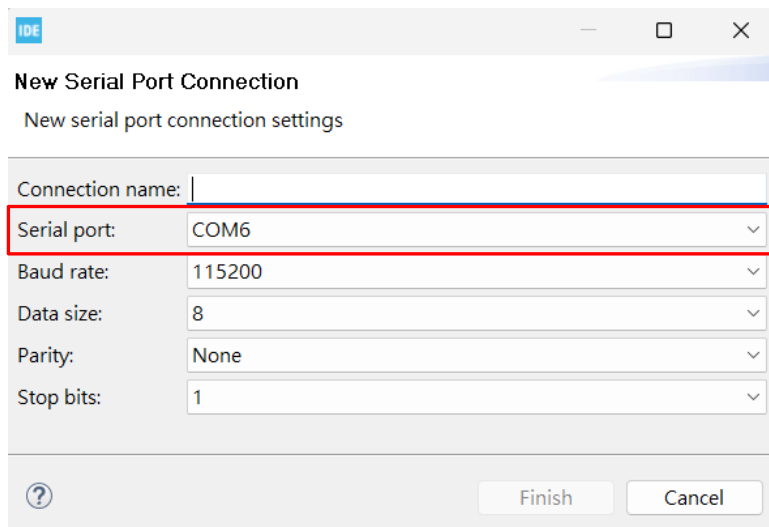
Command shell Set Up

The **serial port** is the USB port number on the PC.



How to find serial port?

1. 開啟裝置管理員
2. USB-to-TTL Serial Adapter插上電腦時，裝置管理員的**連接埠**會跳出一個**Prolific USB-to-Serial Comm port**
3. **Serial port**就會是括弧裡的**編號**



OS HAL UART Transmit

```
HAL_StatusTypeDef HAL_UART_Transmit(  
    UART_HandleTypeDef *huart,  
    const uint8_t *pData,  
    uint16_t Size,  
    uint32_t Timeout);
```

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **pData:** Pointer to data buffer.
- **Size:** Amount of data elements to be sent.
- **Timeout:** Timeout duration.

In **Drivers\STM32F4xx_HAL_Driver\Src\stm32f4xx_hal_uart.c**

OUTLINE

01 Lab requirement

02 UART

03 Lab 2

Add new code

- Create 4 tasks:
 - **Red_LED_App**
 - priority = 1
 - **Green_LED_App**
 - priority = 1
 - **Delay_App**
 - priority = 14
 - **TaskMonitor_App**
 - priority = 3 (Set the *uxStackDepth* to at least 256 words.)

Add new code

- **task.h**

- #include “stm32f4xx_hal.h”
- UART_HandleTypeDef huart2; // for USART2
- void Taskmonitor(void);

- **tasks.c**

- void Taskmonitor(void){...}

- **list.h**

- (Optional) useful macro
 - #define listGET_ITEM_OF_HEAD_ENTRY(pxList) ((&((pxList)->xListEnd))->pxNext)
 - #define listLIST_IS_EMPTY(pxList) (((pxList)->uxNumberOfItems == (UBaseType_t) 0) ? pdTRUE : pdFALSE)
 - Usage: listLIST_IS_EMPTY(&(pxReadyTasksLists[xPriority]));
 - #define listCURRENT_LIST_LENGTH(pxList) ((pxList)->uxNumberOfItems)
 - Usage: listCURRENT_LIST_LENGTH(&(pxReadyTasksLists[xPriority]));

- **FreeRTOSConfig.h**

- configMAX_PRIORITIES **15**

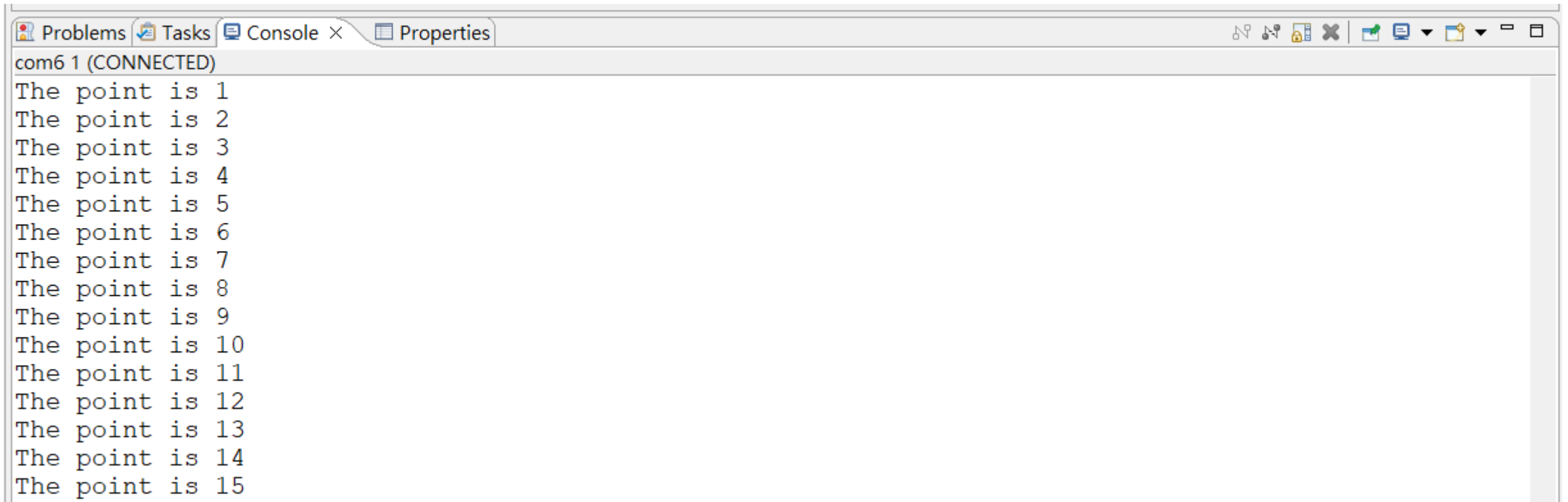
OS USART TX Test

Create a task to test USART.

```
UART_HandleTypeDef huart2;
void USART_Test(void *pvParameters){
    uint32_t Monitortimer = 400;
    char MonitorTest[30];
    char num[15];
    int i = 0;
    while(1){
        memset(MonitorTest, '\0', sizeof(MonitorTest));
        memset(num, '\0', sizeof(num));
        itoa(i, num, 10);
        strcat(num, " ");
        sprintf(MonitorTest, "The point is %s\n\r", num);
        HAL_UART_Transmit(&huart2, (uint8_t *)MonitorTest, strlen(MonitorTest), 0xffff);
        vTaskDelay(Monitortimer);
        Monitortimer += 1;
        i += 1;
    }
}
```

OS USART TX Test

If all settings are correct, you should see console output similar to the picture below after running. Only a single console connection to the USB-to-TTL adapter's port is required.

A screenshot of an IDE's console window. The window has a title bar with tabs for 'Problems', 'Tasks', 'Console', and 'Properties'. The 'Console' tab is active, showing a connection to 'com6 1 (CONNECTED)'. The console output consists of 15 lines, each starting with 'The point is' followed by a number from 1 to 15. The text is displayed in a monospaced font with a light blue background.

```
com6 1 (CONNECTED)
The point is 1
The point is 2
The point is 3
The point is 4
The point is 5
The point is 6
The point is 7
The point is 8
The point is 9
The point is 10
The point is 11
The point is 12
The point is 13
The point is 14
The point is 15
```

os Add new code – tasks.c

```
void Taskmonitor(void)
{
    /* Initialize string */
    char Monitor_data[130];
    memset(Monitor_data, '\0', sizeof(Monitor_data));

    /* Stop scheduler */
    /* Taskmonitor() will block when UART is transmitting data */
    /* Scheduler will change list data when Taskmonitor() is blocked */
    vTaskSuspendAll();

    /* Print title */
    sprintf(Monitor_data, "[Name |Priority(Base/actual) |pxStack |pxTopOfStack |State |n\r");
    HAL_UART_Transmit(&huart2, (uint8_t *)Monitor_data, strlen(Monitor_data), 0xffff);

    /* pxReadyTasksLists */
    /* pxDelayedTaskList */
    /* pxOverflowDelayedTaskList */

    /* Resume scheduler */
    xTaskResumeAll();
}
```

Traverse the lists and print
the TCB information.

Add new code – main.c

```
void TaskMonitor_App(void *pvParameters){
    for(;;){
        Taskmonitor();
        vTaskDelay(1000);
    }
}

void Red_LED_App(void *pvParameters){
    uint32_t Redtimer = 800;
    for(;;){
        HAL_GPIO_TogglePin(GPIOD, Red_LED_Pin);
        vTaskDelay(Redtimer);
        Redtimer+=1;
    }
}
```

Add new code – main.c

```
void Green_LED_App(void *pvParameters){
    uint32_t Greentimer = 1000;
    for(;;){
        HAL_GPIO_TogglePin(GPIOD, Green_LED_Pin);
        vTaskDelay(Greentimer);
        Greentimer+=2;
    }
}

void Delay_App(void *pvParameters){
    int delayflag=0;
    uint32_t delaytime;
    while(1){
        if(delayflag==0){
            delaytime = 1000;
            delayflag=1;
        }else{
            delaytime=0xFFFFFFFF;
        }
        vTaskDelay(delaytime);
    }
}
```

```
340 PRIVILEGED_DATA static List_t pxReadyTasksLists[ configMAX_PRIORITIES ]; /*< Prioritised ready tasks. */
341 PRIVILEGED_DATA static List_t xDelayedTaskList1; /*< Delayed tasks. */
342 PRIVILEGED_DATA static List_t xDelayedTaskList2; /*< Delayed tasks (two lists are used - one for delays that have overflowed the current tick count. */
343 PRIVILEGED_DATA static List_t * volatile pxDelayedTaskList; /*< Points to the delayed task list currently being used. */
344 PRIVILEGED_DATA static List_t * volatile pxOverflowDelayedTaskList; /*< Points to the delayed task list currently being used to hold tasks that have overflowed the current tick count. */
```

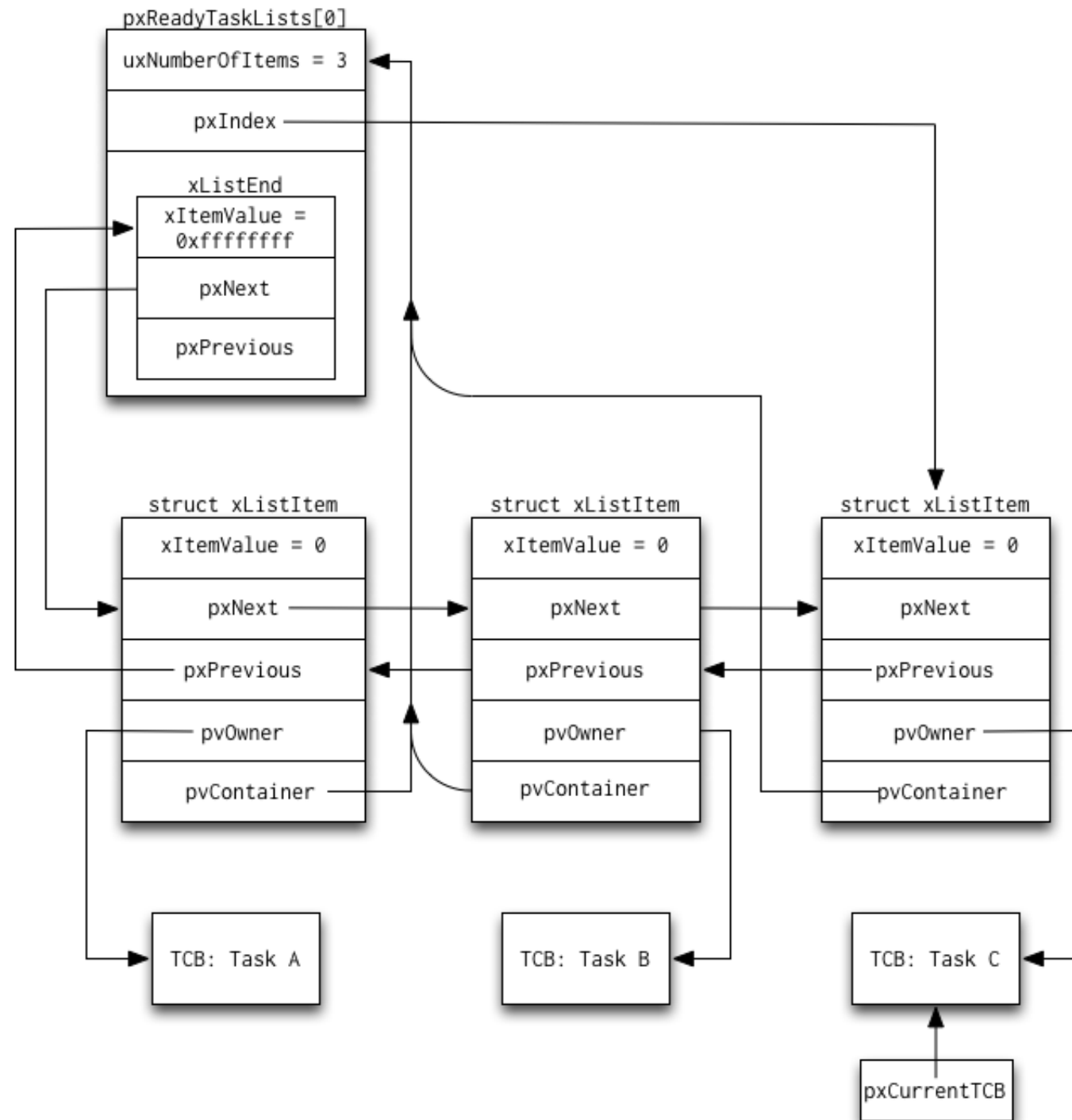
Use the above lists to get the TCB information.

We only need to access:

1. pxReadyTasksLists
2. pxDelayedTaskList
3. pxOverflowDelayedTaskList

OS List

Walk through the list to access each TCB.



OS list.h - xLIST_ITEM

```
147 struct xLIST_ITEM
148 {
149     listFIRST_LIST_ITEM_INTEGRITY_CHECK_VALUE
150     configLIST_VOLATILE TickType_t xItemValue;
151     struct xLIST_ITEM * configLIST_VOLATILE pxNext;
152     struct xLIST_ITEM * configLIST_VOLATILE pxPrevious;
153     void * pvOwner;
154     struct xLIST * configLIST_VOLATILE pxContainer;
155     listSECOND_LIST_ITEM_INTEGRITY_CHECK_VALUE
156 };
157 typedef struct xLIST_ITEM ListItem_t;
```


OS tasks.c - Task Control Block(TCB)

```
247 /*
248 * Task control block. A task control block (TCB) is allocated for each task,
249 * and stores task state information, including a pointer to the task's context
250 * (the task's run time environment, including register values)
251 */
252 typedef struct tskTaskControlBlock /* The old naming convention is used to prevent breaking kernel aware debuggers. */
253 {
254     volatile StackType_t *pxTopOfStack; /*< Points to the location of the last item placed on the tasks stack. THIS MUST BE THE FIRST MEMBER OF THE TCB STRUCT. */
255
256     #if ( portUSING_MPU_WRAPPERS == 1 )
257         xMPU_SETTINGS xMPUSettings; /*< The MPU settings are defined as part of the port layer. THIS MUST BE THE SECOND MEMBER OF THE TCB STRUCT. */
258     #endif
259
260     ListItem_t xStateListItem; /*< The list that the state list item of a task is reference from denotes the state of that task (Ready, Blocked, Suspended ). */
261     ListItem_t xEventListItem; /*< Used to reference a task from an event list. */
262     UBaseType_t uxPriority; /*< The priority of the task. 0 is the lowest priority. */
263     StackType_t *pxStack; /*< Points to the start of the stack. */
264     char pcTaskName[ configMAX_TASK_NAME_LEN ]; /*< Descriptive name given to the task when created. Facilitates debugging only. */ /*lint !e971 Unqualified
265
266     #if ( ( portSTACK_GROWTH > 0 ) || ( configRECORD_STACK_HIGH_ADDRESS == 1 ) )
267         StackType_t *pxEndOfStack; /*< Points to the highest valid address for the stack. */
268     #endif
269
270     #if ( portCRITICAL_NESTING_IN_TCB == 1 )
271         UBaseType_t uxCriticalNesting; /*< Holds the critical section nesting depth for ports that do not maintain their own count in the port layer. */
272     #endif
273
274     #if ( configUSE_TRACE_FACILITY == 1 )
275         UBaseType_t uxTCBNumber; /*< Stores a number that increments each time a TCB is created. It allows debuggers to determine when a task has been deleted
276         UBaseType_t uxTaskNumber; /*< Stores a number specifically for use by third party trace code. */
277     #endif
278
279     #if ( configUSE_MUTEXES == 1 )
280         UBaseType_t uxBasePriority; /*< The priority last assigned to the task - used by the priority inheritance mechanism. */
281         UBaseType_t uxMutexesHeld;
282     #endif
283
284     #if ( configUSE_APPLICATION_TASK_TAG == 1 )
285         TaskHookFunction_t pxTaskTag;
286     #endif
287
288     #if( configNUM_THREAD_LOCAL_STORAGE_POINTERS > 0 )
289         void *pvThreadLocalStoragePointers[ configNUM_THREAD_LOCAL_STORAGE_POINTERS ];
290     #endif
```

OS tasks.c - Task Control Block(TCB)

```
291
292     #if( configGENERATE_RUN_TIME_STATS == 1 )
293     |     uint32_t         ulRunTimeCounter;    /*< Stores the amount of time the task has spent in the Running state. */
294     #endif
295
296     #if ( configUSE_NEWLIB_REENTRANT == 1 )
297     |     /* Allocate a Newlib reent structure that is specific to this task.
298     |     Note Newlib support has been included by popular demand, but is not
299     |     used by the FreeRTOS maintainers themselves. FreeRTOS is not
300     |     responsible for resulting newlib operation. User must be familiar with
301     |     newlib and must provide system-wide implementations of the necessary
302     |     stubs. Be warned that (at the time of writing) the current newlib design
303     |     implements a system-wide malloc() that must be provided with locks. */
304     |     struct _reent xNewLib_reent;
305     #endif
306
307     #if( configUSE_TASK_NOTIFICATIONS == 1 )
308     |     volatile uint32_t ulNotifiedValue;
309     |     volatile uint8_t ucNotifyState;
310     #endif
311
312     /* See the comments in FreeRTOS.h with the definition of
313     tskSTATIC_AND_DYNAMIC_ALLOCATION_POSSIBLE. */
314     #if( tskSTATIC_AND_DYNAMIC_ALLOCATION_POSSIBLE != 0 ) /*lint !e731 !e9029 Macro has been consolidated for readability reasons. */
315     |     uint8_t ucStaticallyAllocated;    /*< Set to pdTRUE if the task is a statically allocated to ensure no attempt is made to free the memory. */
316     #endif
317
318     #if( INCLUDE_xTaskAbortDelay == 1 )
319     |     uint8_t ucDelayAborted;
320     #endif
321
322     #if( configUSE_POSIX_ERRNO == 1 )
323     |     int iTskErrno;
324     #endif
325
326 } tskTCB;
```

tasks.c - prvAddCurrentTaskToDelayedList

```
static void prvAddCurrentTaskToDelayedList( TickType_t xTicksToWait, const BaseType_t xCanBlockIndefinitely )
{
....
    /* Calculate the time at which the task should be woken if the event
    does not occur. This may overflow but this doesn't matter, the
    kernel will manage it correctly. */
    xTimeToWake = xConstTickCount + xTicksToWait;

    /* The list item will be inserted in wake time order. */
    listSET_LIST_ITEM_VALUE( &(amp; pxCurrentTCB->xStateListItem ), xTimeToWake );

    if( xTimeToWake < xConstTickCount )
    {
        /* Wake time has overflowed. Place this item in the overflow
        list. */
        vListInsert( pxOverflowDelayedTaskList, amp; pxCurrentTCB->xStateListItem );
    }
    else
    {
        /* The wake time has not overflowed, so the current block list
        is used. */
        vListInsert( pxDelayedTaskList, amp; pxCurrentTCB->xStateListItem );
    }
...
}
```

Reference

- [Discovery kit with STM32F407VG MCU - User manual](#)
- [Getting started with UART](#)